

Aliyah Khan

9/9/2026

COP4045 - Homework 1

Problem 1

```
import math
import matplotlib.pyplot as plt
import numpy as np

#q1
def quadratic(a, b, c):
    a = float(a)
    b = float(b)
    c = float(c)
    #calculating under radical
    radical = (b**2)-(4*a*c)
    if radical < 0:
        print("no real solutions")
    else:
        #calculating entire formula if possible
        plus = ((-b) + (math.sqrt(radical)))/(2*a)
        minus = ((-b) - (math.sqrt(radical)))/(2*a)
        if radical == 0:
            print(f"one solution: {plus:.5f}")
        if radical > 0:
            print(f"two solutions: x1={minus:.5f} x2 ={plus:.5f}")
    return a, b, c, radical

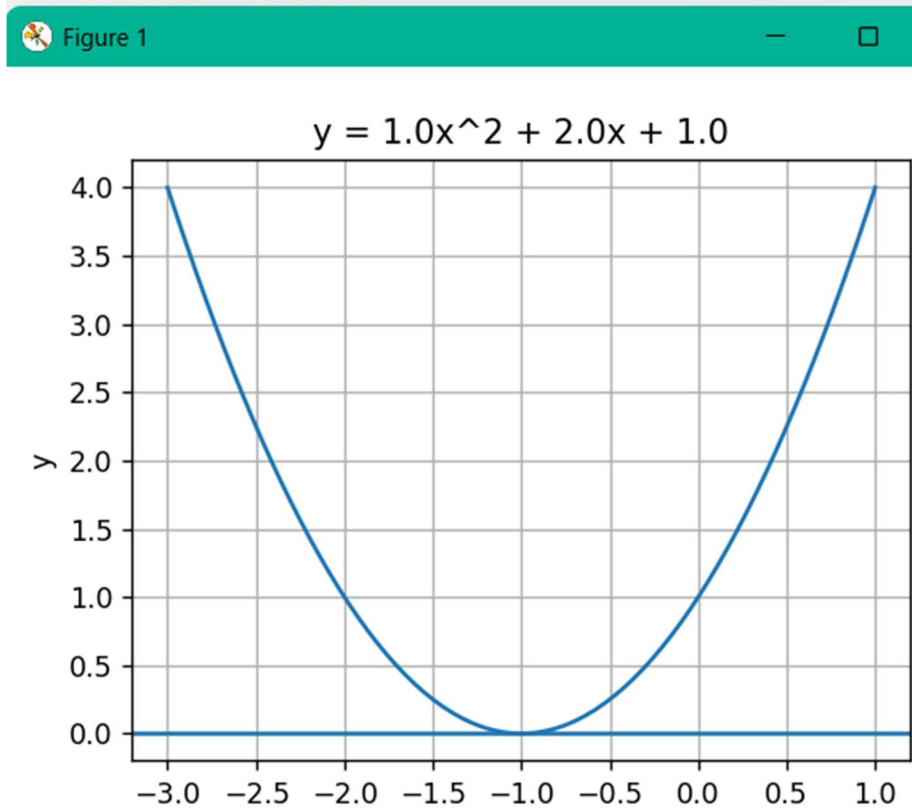
while True:
    a = input("Enter a: ")
    if a == "":
        break
    b = input("Enter b: ")
    c = input("Enter c: ")
    a, b, c, radical = quadratic(a, b, c)
    print()

    if radical < 0:
        #picking domain for no real solutions
        xopt = (-b)/(2*a)
        xmin = xopt - 5
        xmax = xopt + 5
    else:
        #picking domain for one or two solutions
        x1 = ((-b) - math.sqrt(radical))/(2*a)
        x2 = ((-b) + math.sqrt(radical))/(2*a)
        xmin = min(x1, x2) - 2
        xmax = max(x1, x2) + 2
```

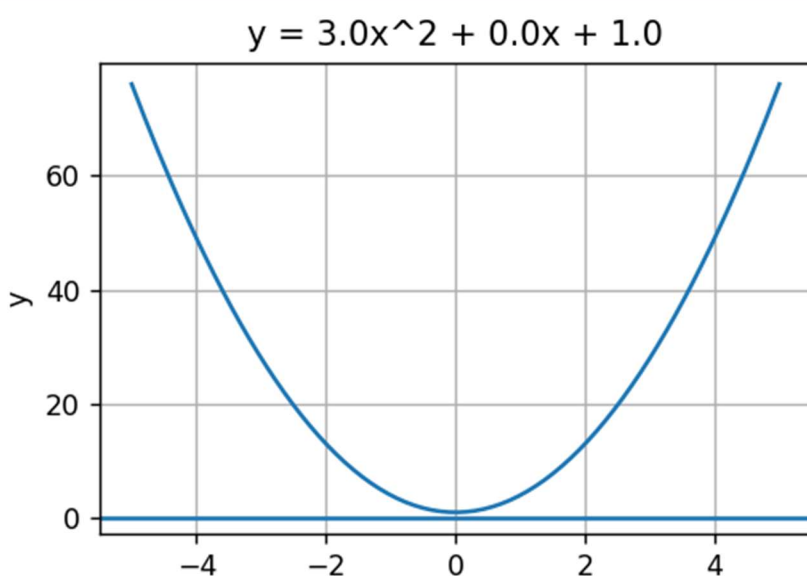
```
x = np.linspace(xmin, xmax, 150)
y = a*x**2 + b*x + c

#graphing
plt.figure()
plt.plot(x, y)
plt.axhline(0)
plt.xlabel("x")
plt.ylabel("y")
plt.title(f"y = {a}x^2 + {b}x + {c}")
plt.grid()
plt.show()

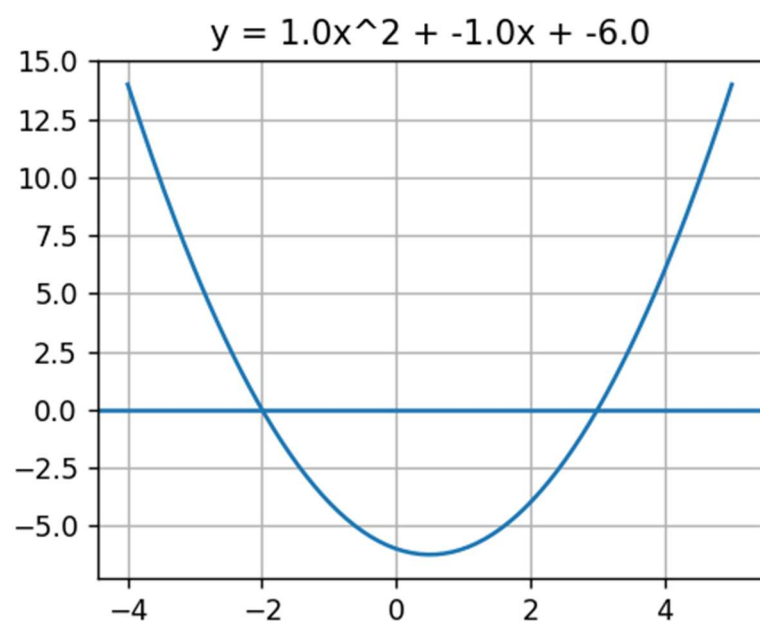
print()
Enter a: 1
Enter b: 2
Enter c: 1
one solution: -1.00000
```



```
Enter a: 3
Enter b: 0
Enter c: 1
no real solutions
```



```
Enter a: 1
Enter b: -1
Enter c: -6
two solutions: x1=-2.00000 x2 =3.00000
```



Problem 2

```
#2
def find_Pythagorean(n):
    triplesList = []
    triples = ()

    #iterating through a, b, and c, each up to n to see if the pythagoream
    #theorem is true for a set of numbers and adding those numbers to the
    #list of tuples
    #using nested for loops ensures cases like (3,4,5) and (4,3,5) are
    #included
    for a in range(1, n+1):
        for b in range(1, n+1):
            for c in range(1, n+1):
                if ((a**2 + b**2) == (c**2)):
                    triplesList.append((a,b,c))
    print("\nList of pythagorean triples: ", triplesList, sep = "")

n = int(input("Enter n: "))
find_Pythagorean(n)
```

Problem 3

```
#3
def find_dup_str(s, n):
    #checking every possible first substring
    for i in range(len(s) - n + 1):
        sub = s[i:i+n]
        #starting after the first substring to avoid overlap
        for j in range(i + n, len(s) - n + 1):
            if sub == s[j:j+n]:
                return sub
    return ""

def find_max_dup(s):
    #starting with the longest possible substring and going down
    for n in range(len(s), 0, -1):
        #calling function above to see if a duplicate of the
        #same length exists
        result = find_dup_str(s, n)
        if result != "":
            return result
    return ""

s = input("Enter a string: ")
n = int(input("Enter substring length: "))

print("\nDuplicated substring of length ", n, ": ", find_dup_str(s, n), sep =
"")
```

```
print("Longest duplicated substring:", find_max_dup(s))
```

Problem 4

```
#4
```

```
import math
```

```
import matplotlib.pyplot as plt
```

```
def plot_function(fun_str, domain, ns):  
    xmin, xmax = domain
```

```
    #creating a list of x values by calculating  
    #average distance between each point
```

```
    xlist = []  
    split = (xmax - xmin) / (ns - 1)  
    for i in range(ns):  
        x = xmin + i * split  
        xlist.append(x)
```

```
    #calculating y values by inputting x  
    #into the user's formula
```

```
    ylist = []  
    for x in xlist:  
        y = eval(fun_str)  
        ylist.append(y)
```

```
    #printing the table of points
```

```
    print("\n          x          y")  
    print("-----")
```

```
    for i in range(min(12, len(xlist))):  
        print(f"{xlist[i]:10.4f} {ylist[i]:10.4f}")  
    if len(xlist) > 12:  
        print("----- (rest of the table not shown)")  
    else:  
        print("-----")
```

```
    #making graph
```

```
    plt.plot(xlist, ylist, "o-")  
    plt.xlabel("x")  
    plt.ylabel("y")  
    plt.title(fun_str)  
    plt.grid()  
    plt.show()
```

```
fun_str = input("Enter function with variable x: ")  
ns = int(input("Enter number of samples: "))  
xmin = float(input("Enter xmin: "))  
xmax = float(input("Enter xmax: "))  
domain = (xmin, xmax)  
plot_function(fun_str, domain, ns)
```

Enter function with variable x: `2 * math.sin(2*math.pi*x)`

100

-3

+3

Enter number of samples: Enter xmin: Enter xmax:

x	y
---	---

-3.0000	0.0000
---------	--------

-2.9394	0.7433
---------	--------

-2.8788	1.3802
---------	--------

-2.8182	1.8193
---------	--------

-2.7576	1.9977
---------	--------

-2.6970	1.8900
---------	--------

-2.6364	1.5115
---------	--------

-2.5758	0.9165
---------	--------

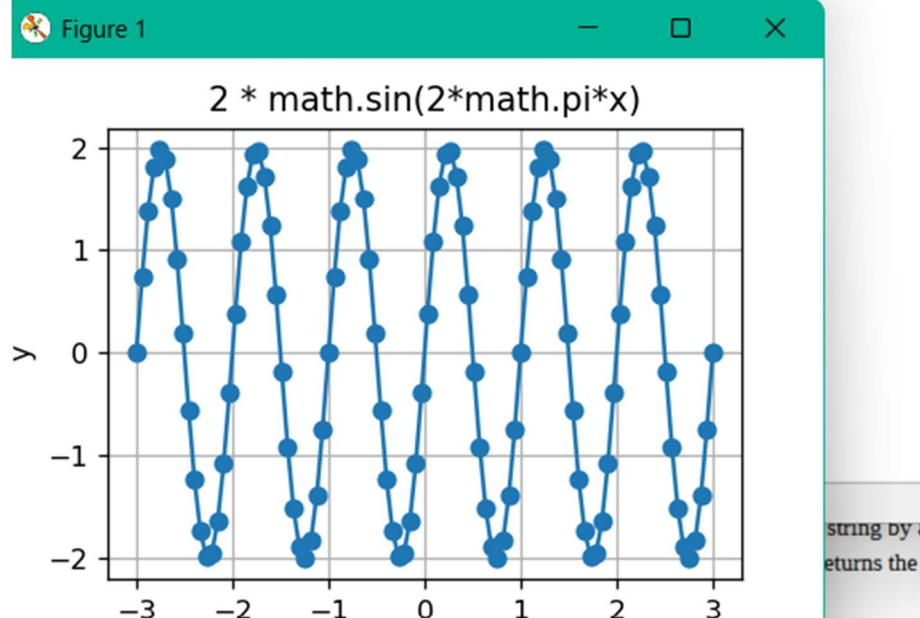
-2.5152	0.1901
---------	--------

-2.4545	-0.5635
---------	---------

-2.3939	-1.2363
---------	---------

-2.3333	-1.7321
---------	---------

(rest of the table not shown)



Problem 5

```
def caesar_cipher(text, shift):  
    result = ""  
  
    # go through each character  
    for i in range(len(text)):  
        char = text[i]  
  
        if char.isalpha():  
            # get the alphabet position
```

```

        if char.isupper():
            start = ord("A")
        else:
            start = ord("a")

        # shift the letter and wrap around the alphabet
        new_char = chr((ord(char) - start + shift) % 26 + start)
        result += new_char
    else:
        result += char

    return result

def caesar_decipher(cyphertext, shift):
    # decrypt by shifting in the opposite direction
    return caesar_cipher(cyphertext, -shift)

def letter_frequency(text):
    frequency = {}

    # create an entry for every letter
    for i in range(26):
        letter = chr(ord("a") + i)
        frequency[letter] = 0

    # count letters in the text
    for i in range(len(text)):
        char = text[i].lower()

        if char.isalpha():
            frequency[char] += 1

    return frequency

def main():
    text = ""
    shift = 0

    while True:
        print("\n--- Caesar Cipher Menu ---")
        print("1. Enter a message")
        print("2. Enter a shift value")
        print("3. Encrypt message")
        print("4. Show letter frequency")
        print("5. Decrypt message")
        print("6. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":

```

```

    text = input("Enter your message: ")
    print("message saved.")

elif choice == "2":
    try:
        shift = int(input("Enter the shift value: "))
        print("shift saved.")
    except ValueError:
        print("please enter an integer.")

elif choice == "3":
    if text == "":
        print("please enter a message first.")
    else:
        encrypted = caesar_cipher(text, shift)
        print("encrypted text:", encrypted)

elif choice == "4":
    if text == "":
        print("please enter a message first.")
    else:
        frequency = letter_frequency(text)
        print("letter frequency:")

        for letter in frequency:
            print(letter + ":", frequency[letter])

elif choice == "5":
    if text == "":
        print("please enter a message first.")
    else:
        encrypted = caesar_cipher(text, shift)
        decrypted = caesar_decipher(encrypted, shift)
        print("encrypted text:", encrypted)
        print("decrypted text:", decrypted)

elif choice == "6":
    print("goodbye!")
    break

else:
    print("invalid choice. please try again.")

if __name__ == "__main__":
    main()

```

Problem 5 Test code

```

import unittest
from p5_Khan_Aliyah import caesar_cipher, caesar_decipher, letter_frequency

```



```

class TestCaesarCipher(unittest.TestCase):

    def test_cipher(self):
        self.assertEqual(caesar_cipher("hello", 3), "khoor")

    def test_cipher_uppercase(self):
        self.assertEqual(caesar_cipher("Hello World!", 3), "Khoor Zruog!")

    def test_cipher_wraparound(self):
        self.assertEqual(caesar_cipher("xyz", 3), "abc")

    def test_cipher_negative_shift(self):
        self.assertEqual(caesar_cipher("abc", -3), "xyz")

class TestCaesarDecipher(unittest.TestCase):

    def test_decipher(self):
        encrypted = caesar_cipher("Hello World!", 5)
        self.assertEqual(caesar_decipher(encrypted, 5), "Hello World!")

class TestLetterFrequency(unittest.TestCase):

    def test_frequency(self):
        result = letter_frequency("Hello!")
        self.assertEqual(result["h"], 1)
        self.assertEqual(result["e"], 1)
        self.assertEqual(result["l"], 2)
        self.assertEqual(result["o"], 1)

    def test_ignores_spaces_and_symbols(self):
        result = letter_frequency("A B-C!")
        self.assertEqual(result["a"], 1)
        self.assertEqual(result["b"], 1)
        self.assertEqual(result["c"], 1)
        self.assertEqual(result["d"], 0)

if __name__ == "__main__":
    unittest.main()

```

Write a program in a file p5_.....py that interacts with the user via the terminal to encrypt/decrypt text and analyze letter frequencies.

a) Write a function `caesar_cipher(text, shift)` that shifts letters in a string by a given integer using string indexing and loops, preserving spaces and casing, and that returns the shifted (encrypted) string.

b) Write a function `caesar_decipher(cyphertext, shift)` that decrypts a Caesar-encrypted string and that return the original string clear string.

c) Write a function `letter_frequency(text)` that counts how many times each letter of the alphabet appears in the text, ignoring case and non-alphabetic characters.

d) Create a `main()` function with a terminal menu allowing the user to type a message, specify a shift value, and view both the ciphered text, the letter frequency breakdown, and the deciphered text.

e) Write a separate test file `p5_<your name>_test.py` using Python's `unittest` framework to verify your cipher and frequency functions.

Show less ^



</> Python

```
from pathlib import Path
```

```
program = '''def caesar_cipher(text, shift):
```

```
    result = ""
```

```
    # go through each character
```

```
    for i in range(len(text)):
```

```
        char = text[i]
```

```
        if char.isalpha():
```

```
            # get the alphabet position
```

```
            if char.isupper():
```

```
                start = ord("A")
```

```
            else:
```

```
                start = ord("a")
```

```
            # shift the letter and wrap around the alphabet
```

```
            new_char = chr((ord(char) - start + shift) % 26 + start)
```

```
            result += new_char
```

```
        else:
```

```
            result += char
```

```
    return result
```

```
def caesar_decipher(cyphertext, shift):
    # decrypt by shifting in the opposite direction
    return caesar_cipher(cyphertext, -shift)

def letter_frequency(text):
    frequency = {}

    # create an entry for every letter
    for i in range(26):
        letter = chr(ord("a") + i)
        frequency[letter] = 0

    # count letters in the text
    for i in range(len(text)):
        char = text[i].lower()

        if char.isalpha():
            frequency[char] += 1

    return frequency
```

```
def main():
    text = ""
    shift = 0

    while True:
        print("\n--- Caesar Cipher Menu ---")
        print("1. Enter a message")
        print("2. Enter a shift value")
        print("3. Encrypt message")
        print("4. Show letter frequency")
        print("5. Decrypt message")
        print("6. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            text = input("Enter your message: ")
            print("message saved.")

        elif choice == "2":
            try:
                shift = int(input("Enter the shift value: "))
                print("shift saved.")
            except ValueError:
                print("please enter an integer.")
```

```
elif choice == "3":
    if text == "":
        print("please enter a message first.")
    else:
        encrypted = caesar_cipher(text, shift)
        print("encrypted text:", encrypted)

elif choice == "4":
    if text == "":
        print("please enter a message first.")
    else:
        frequency = letter_frequency(text)
        print("letter frequency:")

        for letter in frequency:
            print(letter + ":", frequency[letter])

elif choice == "5":
    if text == "":
        print("please enter a message first.")
    else:
        encrypted = caesar_cipher(text, shift)
        decrypted = caesar_decipher(encrypted, shift)
        print("encrypted text:", encrypted)
        print("decrypted text:", decrypted)
```

```
        elif choice == "6":
            print("goodbye!")
            break

        else:
            print("invalid choice. please try again.")

if __name__ == "__main__":
    main()
...
```

```

test = '''import unittest
from p5_yourname import caesar_cipher, caesar_decipher, letter_frequency

class TestCaesarCipher(unittest.TestCase):

    def test_cipher(self):
        self.assertEqual(caesar_cipher("hello", 3), "khoor")

    def test_cipher_uppercase(self):
        self.assertEqual(caesar_cipher("Hello World!", 3), "Khoor Zruog!")

    def test_cipher_wraparound(self):
        self.assertEqual(caesar_cipher("xyz", 3), "abc")

    def test_cipher_negative_shift(self):
        self.assertEqual(caesar_cipher("abc", -3), "xyz")

class TestCaesarDecipher(unittest.TestCase):

    def test_decipher(self):
        encrypted = caesar_cipher("Hello World!", 5)
        self.assertEqual(caesar_decipher(encrypted, 5), "Hello World!")

```

```

class TestLetterFrequency(unittest.TestCase):

    def test_frequency(self):
        result = letter_frequency("Hello!")
        self.assertEqual(result["h"], 1)
        self.assertEqual(result["e"], 1)
        self.assertEqual(result["l"], 2)
        self.assertEqual(result["o"], 1)

    def test_ignores_spaces_and_symbols(self):
        result = letter_frequency("A B-C!")
        self.assertEqual(result["a"], 1)
        self.assertEqual(result["b"], 1)
        self.assertEqual(result["c"], 1)
        self.assertEqual(result["d"], 0)

```

```

if __name__ == "__main__":
    unittest.main()
...

program_path = Path("/mnt/data/p5_yourname.py")
test_path = Path("/mnt/data/p5_yourname_test.py")

program_path.write_text(program, encoding="utf-8")
test_path.write_text(test, encoding="utf-8")

print(f"Created: {program_path}")
print(f"Created: {test_path}")

```

STDOUT/STDERR

✦ Upgrade 

Created: /mnt/data/p5_yourname.py
 Created: /mnt/data/p5_yourname_test.py

Here are the two files for the assignment. I kept the code straightforward and used **string indexing and loops** as requested.

-  [Download p5_yourname.py](#)
-  [Download p5_yourname_test.py](#)

What you need to change

Replace `yourname` in the filenames and in the test file's import with your actual name. For example:

p5_alex.py
 p5_alex_test.py



Then change:

 Python



 Run

```
from p5_yourname import caesar_cipher, caesar_decipher, letter_frequency
```

to:



 Python



 Run

```
from p5_alex import caesar_cipher, caesar_decipher, letter_frequency
```

How to run the tests

Put both files in the same folder, then run:

 Bash



```
python -m unittest p5_yourname_test.py
```

You should see something similar to:

.....



Ran 7 tests in 0.001s

OK

The program includes:

- `caesar_cipher()` — encrypts the message
- `caesar_decipher()` — decrypts the message
- `letter_frequency()` — counts each alphabet letter
- `main()` — terminal menu for entering messages, shifts, encryption, frequency analysis, and decryption
- `unittest` tests for the required functions 

 `p5_yourname.py`

Code

 `p5_yourname_test.py`

Code